

Project II: Bin Packing

Will Clark

May 16, 2026

Abstract

This report analyzes the waste generated by five bin packing algorithms: *Next Fit*, *First Fit*, *Best Fit*, *First Fit Decreasing* and *Best Fit Decreasing*.

1 Introduction

The *Bin Packing* problem is an optimization problem wherein items of different sizes must be *packed* into a finite number of *bins* with a certain capacity. Formally, take $I = \{i_1, \dots, i_n\}$ to be a set of n items, where for each $i_k \in I$, a function s

$$\begin{aligned} s: I &\rightarrow (0, 1] \\ i_k &\mapsto s(i_k) \end{aligned}$$

denotes the size of the item. We want to know, for a set of bins $B = \{b_1, \dots, b_m\}$ with initial capacities $c(b_i) = 1$ for all $i \in \{1, \dots, m\}$, the minimum number of bins required to pack all $|I|$ items.

Despite its simple nature, the problem is computationally **NP-Hard**, meaning that its incredibly unlikely a polynomial time algorithm exists to *optimally* solve it. There are, however, many approximation algorithms we can use to *approximate* bin packing, of which five will be discussed in later sections.

1.1 Waste

The goal of each of the experiments is to provide empirical evidence for estimating the *Waste* of a bin-packing algorithm. Let $\mathbb{I}$ denote the set of all sequences of valid items and $\mathbb{A}$ the set of all algorithms.

$$W: (\mathbb{A}, \mathbb{I}) \rightarrow \mathbb{R}^+ \cup \{0\}$$

be the waste of an algorithm, A , where for a set of items I :

$$W(A, I) = |\text{Bins Used by } A| - \sum_{i \in I} s(i)$$

We define waste as to compare approximation algorithms, instead of those that find the optimal solution. This is more valuable to reasoning which of the superceding algorithms is best, since computing an approximation ratio of an algorithm

$$\frac{ALG(I)}{OPT(I)}$$

is contingent on knowing the optimal solution, which takes exponential time to compute.

1.2 Data Structures

Traditional *Best Fit* and *Next Fit* variations run in $\mathcal{O}(n^2)$ time. We utilize a *Zip Zip Tree* to optimize both algorithms. Formally, a *Zip Zip Tree* is a binary search tree where every node has a numeric *rank*, and the tree is max-heap ordered with respect to *rank*.

Zip Trees, in their very nature, are constructed in such a way as to emulate a perfect binary tree. Theory tells us that perfect binary tree's have height $h = \log_2(n)$, for n vertices. *Zip Trees* invert this fact, and let *rank* be a geometric random variable with success probability $p = \frac{1}{2}$. To break collisions, we let *rank* be a tuple (r_1, r_2) where

$$\begin{aligned} r_1 &\sim \text{Geom}\left(\frac{1}{2}\right), & \mathcal{O}(\log \log n) \\ r_2 &\sim \text{Unif}[0, \log^c n], & \mathcal{O}(c \log \log n) \end{aligned}$$

and compare nodes lexicographically. *Zip Trees* support **insert**, **find**, and **delete** with help from the **zip** operation. *Zippping* and *Unzipping* preserve the order and max-heap properties. *Zip Trees* are very efficient and easy to implement, and, arguably more importantly, are isomorphic to skip-lists of which were implemented in *Project 1: Sorting*.

2 Next Fit

Next Fit is the most original heuristic approximation algorithm. It is simply:

- Try to place item i into bin b
- If it fits, put it there.
- If not, start a new bin.

The approximation ratio for *Next Fit* is $SOL = k < 2M$, where M is the optimal number of bins. The *Waste* grows linearly

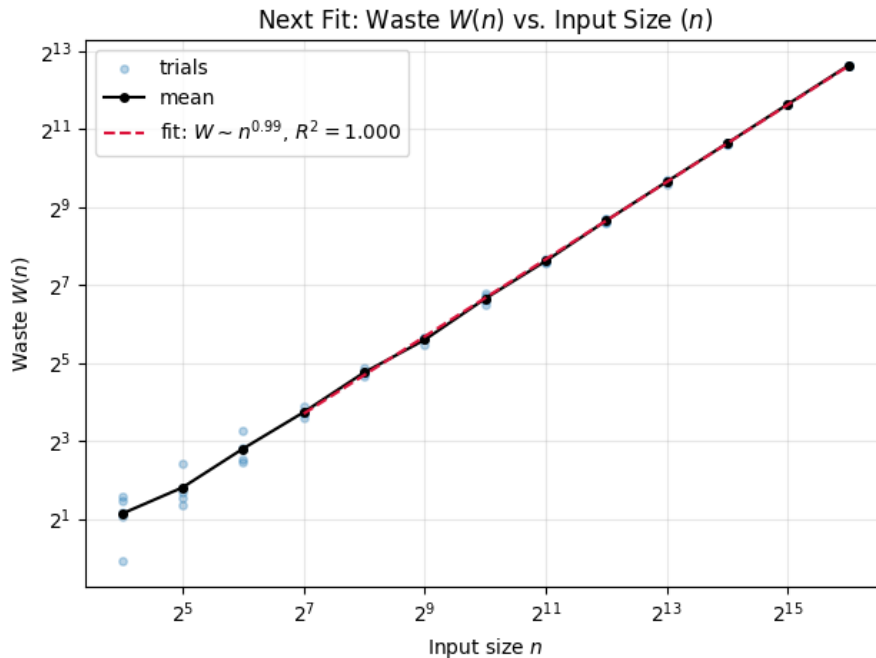


Figure 1: Next Fit Waste

as noted by the figure above. This is in direct agreement with the approximation ratio since there is N waste for an input size of N , meaning we are bounded by N extra buckets in the worst case (all waste requires a new bucket). This is exactly the bound above,

$$W(NF, A) \propto N, \quad SOL = k < 2N$$

3 First Fit

The *First Fit* algorithm is as follows

- Scan through bins $b_1, b_2, \dots b_i$
- Place item i in the first bin that is large enough to hold it, $c(b_i) \geq s(i)$
- Create a new bin b_{i+1} if there are *no* bins that are large enough

From the lecture slides, we know that for M the number of bins to optimally pack I items, the approximation ratio of First Fit, FF , is

$$SOL_{FF} = \lceil 1.7M \rceil.$$

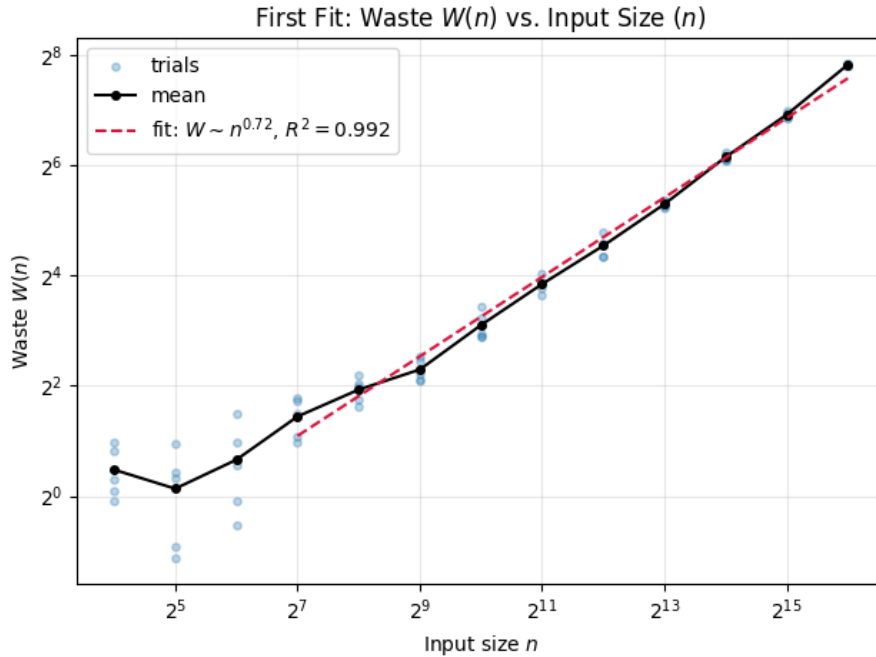


Figure 2: First Fit Waste

We start the best fit line at 2^7 samples to eliminate the noise from smaller inputs. We see that the Waste is of order $\mathcal{O}(n^{0.72})$, which is reflective of it's theoretical result. For a large N , In the worst case, we let each bit of waste require a new bucket, and as such we would be bounded by $N_{OPT} + 0.72N_{FF} = 1.72N$, supporting the theoretical result.

4 Best Fit

Best Fit works as follows:

- For item i , find the bin b^* with the least remaining capacity that still fits:

$$b^* = \arg \min_b \{c(b) - s(i) \mid c(b) \geq s(i)\}$$

where $c(b)$ is the remaining capacity of bin b and $s(i)$ is the size of item i .

- If no such bin exists, open a new one.

The theoretical approximation ratio for *Best Fit*, BF , as suggested by Dosa and Sgall (2014), is

$$SOL_{BF} = k \leq \lfloor 1.70M \rfloor$$

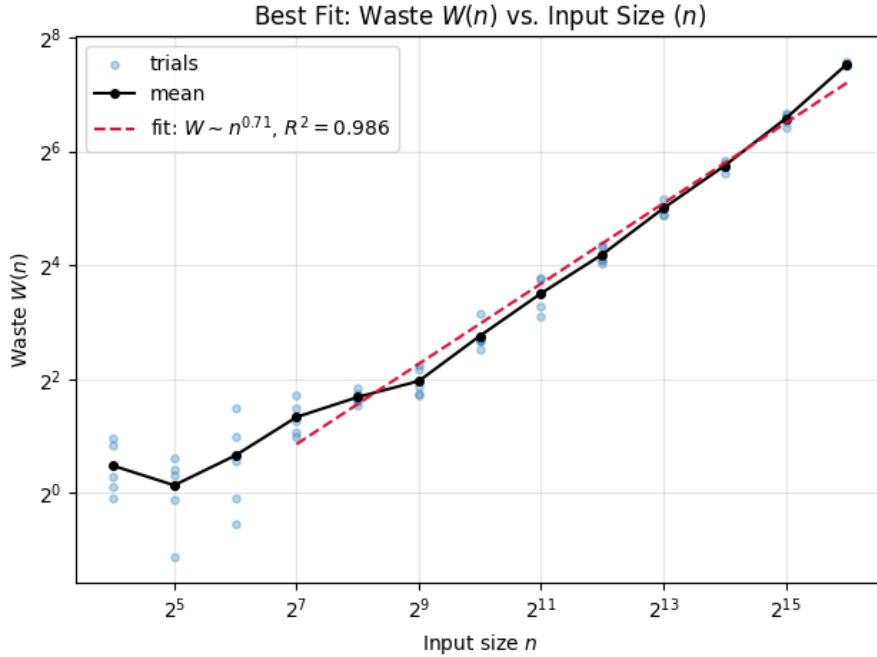


Figure 3: Best Fit Waste

We once again begin fitting the data at 2^7 samples to reduce noise. Notice that there is less variance among the trials (less spread out) than in First Fit, but our R^2 value is lower. This suggests that Best Fit may be a more consistent option than first fit. We see that the empirical waste follows $\mathcal{O}(n^{0.71})$, and by the same reasoning as *Next Fit* and *First Fit*, for a large N support

$$1N_{OPT} + 0.71N_{BF} = 1.71N$$

which is a bit above the suggested lower bound, but close enough to call for statistical variance.

5 First Fit Decreasing

The *First Fit Decreasing* algorithm is

- Sort the items I in descending order

- Run the *First Fit* algorithm [3] on I

Compared to all previous algorithms (*Next Fit*, *First Fit* and *Best Fit*), *First Fit Decreasing* has a wildly different waste-complexity.

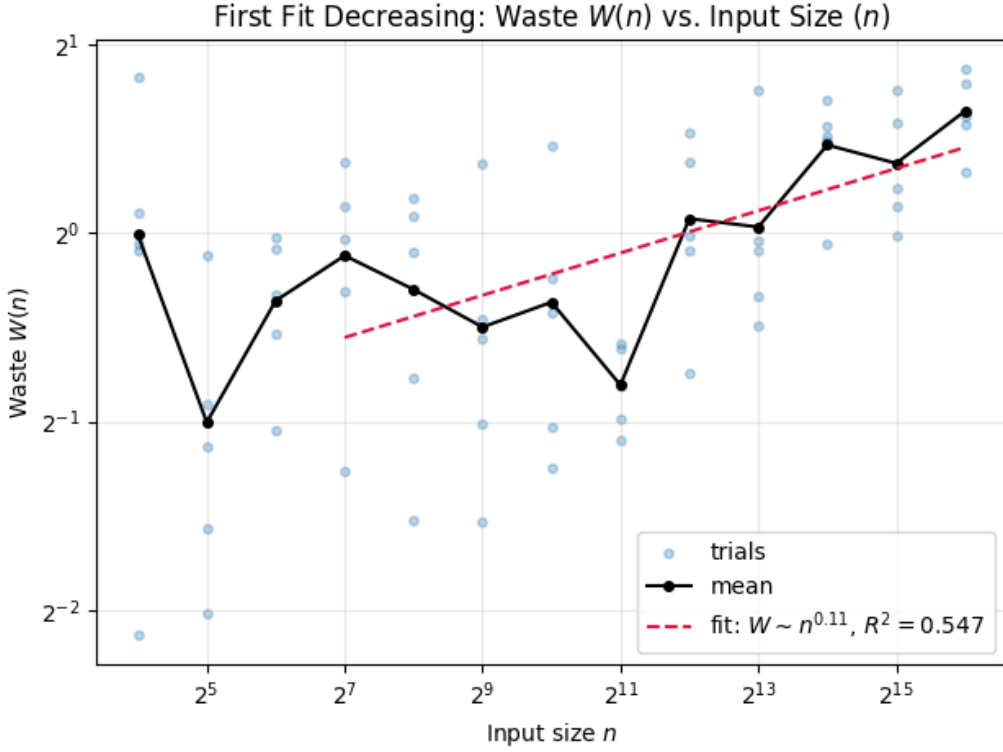


Figure 4: First Fit Decreasing Waste

Note that the scaling of the axis: the trials would appear much closer together if the scaling was the same as the first three algorithms. The empirical waste complexity is $\mathcal{O}(n^{0.11})$, which is much must better than any previous algorithm. Sorting the list in decreasing order lets us more tightly pack (put more items in) the first bins, which has a profound impact on the waste. We should note, however, that *FFD* required us to **sort** the input, which is at best $\mathcal{O}(n \log n)$ using Tim-Sort. While the waste was significantly lower, the runtime might not be.

6 Best Fit Decreasing

The *Best Fit Decreasing* algorithm is

- Sort the items I in descending order
- Run the *Best Fit* algorithm [4] on I

In exactly the same way as *First Fit Decreasing* *Best Fit Decreasing* has a very very low waste-complexity.

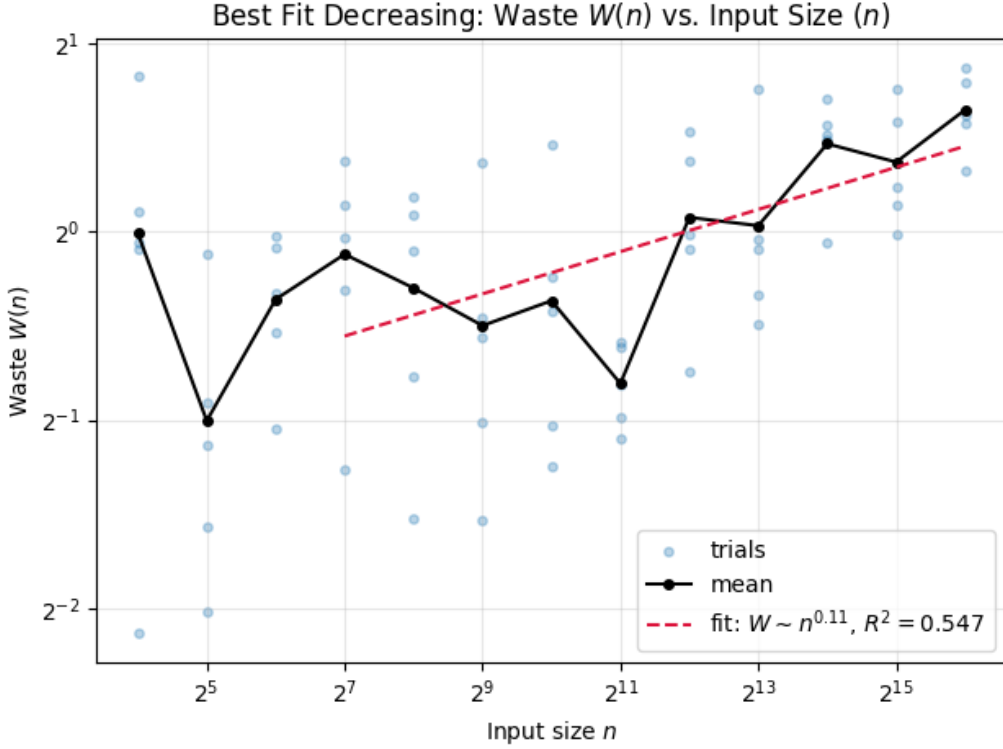


Figure 5: Best Fit Decreasing Waste

Where the empirical waste is order $\mathcal{O}(n^{0.11})$, the same as in *FFD*. Interestingly enough, the plots are *exactly* the same. Even considering the individual trials, both graphs are identical. I conjecture that this is no coincidence. That, in fact, each bin may not be filled the exact same way, but the number of bins is always the same, given the input distribution. (expanded upon in [7]).

7 Conclusion

There are two factors to consider when determining the *best* bin-packing algorithm: *run time* and *waste*.

Let's first rule out *Next Fit* since it's empirical waste complexity was $\mathcal{O}(n^{0.99})$, satisfying its theoretical approximation ratio of $2M$ for M optimal bins. It's simply not a good enough approximation, even though the run-time of the algorithm is the fastest at $\mathcal{O}(n)$.

First fit and *Best fit* perform quite well, with *best fit* slightly edging out the *first fit* with waste complexity $\mathcal{O}_{FF}(n^{0.72})$ vs. $\mathcal{O}_{BF}(n^{0.71})$. This makes sense since *best fit* should, in theory, have tighter packings.

The real decision is between *First fit decreasing* and *best fit decreasing*. Both of the graphs (figures 4, 5) are nearly identical, with empirical waste complexity $\mathcal{O}(n^{0.11})$ in both cases. As conjectured in section [6], this is likely not a coincidence: once the input is sorted in decreasing order, the largest items dominate the packing structure. Since both algorithms have the same waste complexity, we have to pivot to runtime analysis. *Both* algorithms incur a $\mathcal{O}(n \log n)$ sort cost using Tim Sort to sort the items in decreasing order. *BFD* and *FFD* both use a Zip Zip Tree, but the augments are different. Asymptotically, the two are equivalent, but *FFD* may have a smaller constant factor since it accepts the *first* feasible bin, rather than the tightest fit one. As such **First Fit Decreasing** is the best bin-packing algorithm.